Object Oriented Programming Lab (CSEG1044) Experiment-07&08: Classes/Objects + Overloading (Student Handout — Before Submission)

Tofik Ali

March 3, 2026

Contents

1 About this handout **1**

2 Experiment 07 (Syllabus) **1**

3 Experiment 08 (Syllabus) **1**

4 What you must do (minimum requirements) **2**
4.1 Task A — Student class with validation (Exp-07) 2
4.2 Task B — Overloading demo (Exp-08) 2

5 Hints (read only if stuck) **2**

6 What to submit (MANDATORY) **3**

1 About this handout

This handout gives the implementation requirements and submission checklist. Do not copy solutions from outside sources; write and test your own code.

2 Experiment 07 (Syllabus)

- **Objective:** To understand classes, objects, and constructors.
- **Problem Statement:** Design classes with data members and member functions.
- **Expected Outcome:** Students will understand object creation and encapsulation.

3 Experiment 08 (Syllabus)

- **Objective:** To demonstrate compile-time polymorphism.
- **Problem Statement:** Implement method and constructor overloading in Java.

- **Expected Outcome:** Students will understand method resolution at compile time.

4 What you must do (minimum requirements)

4.1 Task A — Student class with validation (Exp-07)

Create `StudentDemo.java` with a `Student` class.

Fields (private):

- `rollNo` (must be positive)
- `name` (must be non-empty)
- `cgpa` (0 to 10)

Requirements:

- Validate in constructor.
- Provide a safe update method `updateCgpa(double)` with validation.
- Override `toString()` to print object details.

4.2 Task B — Overloading demo (Exp-08)

Create `OverloadingDemo.java`:

1. Create class `Box` with overloaded constructors:
   - default: unit box
   - 1 parameter: cube
   - 3 parameters: `w, h, d`
2. Add method `volume()`.
3. Write overloaded methods:
   - `area(double r)` → circle area
   - `area(double w, double h)` → rectangle area
4. In `main`, create objects using different constructors and print volumes and areas.

5 Hints (read only if stuck)

- Keep fields private and expose behavior through methods.
- Constructor overloading can reduce duplicate initialization code using `this(...)` chaining.
- Overloading cannot be done using only return type; parameter list must change.

6 What to submit (MANDATORY)

Submit a single zipped folder containing:

- `StudentDemo.java`
- `OverloadingDemo.java`
- sample output for both programs
- observations (5–10 lines): why encapsulation helps; how Java chooses overloaded methods

Sources: Schildt, *Java: The Complete Reference* (12e), McGraw-Hill (classes, inheritance, packages, interfaces).; Bloch, *Effective Java* (3e), Addison-Wesley, 2018 (design choices; interfaces vs abstract classes).; Oracle Java Tutorials: Classes and Objects + Interfaces + Packages (accessed March 3, 2026).; Java SE API docs (e.g., `Comparable`, `Runnable`) (accessed March 3, 2026).